E2E Recorder v2

User Manual

Version 2.0.0 | Firefox & Chrome Extension

Supported Browsers

- Mozilla Firefox 109+
- Google Chrome 88+
- Microsoft Edge 88+

Supported Frameworks

- Playwright (TypeScript)
- Playwright (Python)
- Cypress (JavaScript)
- Selenium WebDriver (JavaScript)

June 2026

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Key Capabilities 4
1.2 Privacy 4
2. Installation 4
2.1 Firefox 4
2.2 Chrome / Edge 5
3. Popup Overview 5
3.1 Recorder Tab Layout 6
4. Recording a Session 7
4.1 Starting and Stopping 7
4.2 Captured Event Types 7
4.3 Sensitive Data Masking 8
4.4 Traffic-Light Selector Inspector 8
4.5 Multi-Tab Recording 8
5. The Event List 9
5.1 Deleting Events 9
5.2 Reordering Events 9
5.3 Editing a Selector Inline 10
5.4 Inserting New Steps 10
6. Selector Candidates 11
6.1 Opening the Candidates Panel 11
6.2 Candidate Score Dots 11
6.3 Testing a Candidate 12
6.4 Using or Discarding a Candidate 12
7. Assertion Suggestions 13
7.1 Suggestion Types 13
7.2 Accepting or Discarding Suggestions 13
8. Replay (Test Recording) 13
8.1 Running a Replay 13
8.2 Status Indicators 14
9. Exporting Test Scripts 14
9.1 Selecting a Framework 14

9.2 Export Options....15
9.3 Copying or Downloading....15
9.4 Sample: Playwright TypeScript....15
10. Logs Panel....15
10.1 Log Levels....16
10.2 Filtering Logs....16
10.3 Copying and Clearing Logs....16
11. Selector Scoring Reference....17
12. Known Limitations....17
13. Troubleshooting....18
14. Quick Reference Card....19

1. Introduction

E2E Recorder v2 is a browser extension for Firefox and Chrome that captures your interactions with any web application and converts them automatically into production-ready automated end-to-end (E2E) test scripts. No cloud services, no artificial intelligence, and no external dependencies are required — everything runs locally inside your browser.

1.1 Key Capabilities

- Record clicks, text input, keyboard shortcuts, hover actions, scrolling, drag-and-drop, file uploads, and dropdown selections.
- Automatically generate selectors using a stability-scoring engine (`data-testid` > `id` > `aria-label` > …).
- Capture multiple selector alternatives per action and let you choose the best one.
- Multi-tab recording: follow OAuth pop-ups and new-window flows in a single session.
- Shadow DOM and iframe support out of the box.
- Step-by-step replay with per-step pass / fail indicators.
- Export to Playwright TypeScript, Playwright Python, Cypress, or Selenium.
- Page Object Model export (optional).
- Built-in Logs panel for debugging.

1.2 Privacy

All data stays on your device. The extension makes **no** outbound network requests, stores nothing on any server, and never transmits your recorded interactions to any third party. Passwords and credential fields are automatically masked before storage.

2. Installation

2.1 Firefox

1. Build the package: run `build.cmd` (Windows) or `build.sh` (macOS / Linux) from the project root. The output file is `dist/e2e-recorder-v2.xpi`.
2. Open Firefox and navigate to `about:addons`.
3. Click the gear icon (⚙) and choose Install Add-on From File….
4. Select the `.xpi` file and confirm the installation.
5. The E2E Recorder icon appears in the browser toolbar.

> **Tip**
> If Firefox shows a warning that the extension is unsigned, you can load it temporarily via `about:debugging` → This Firefox → Load Temporary Add-on….

2.2 Chrome / Edge

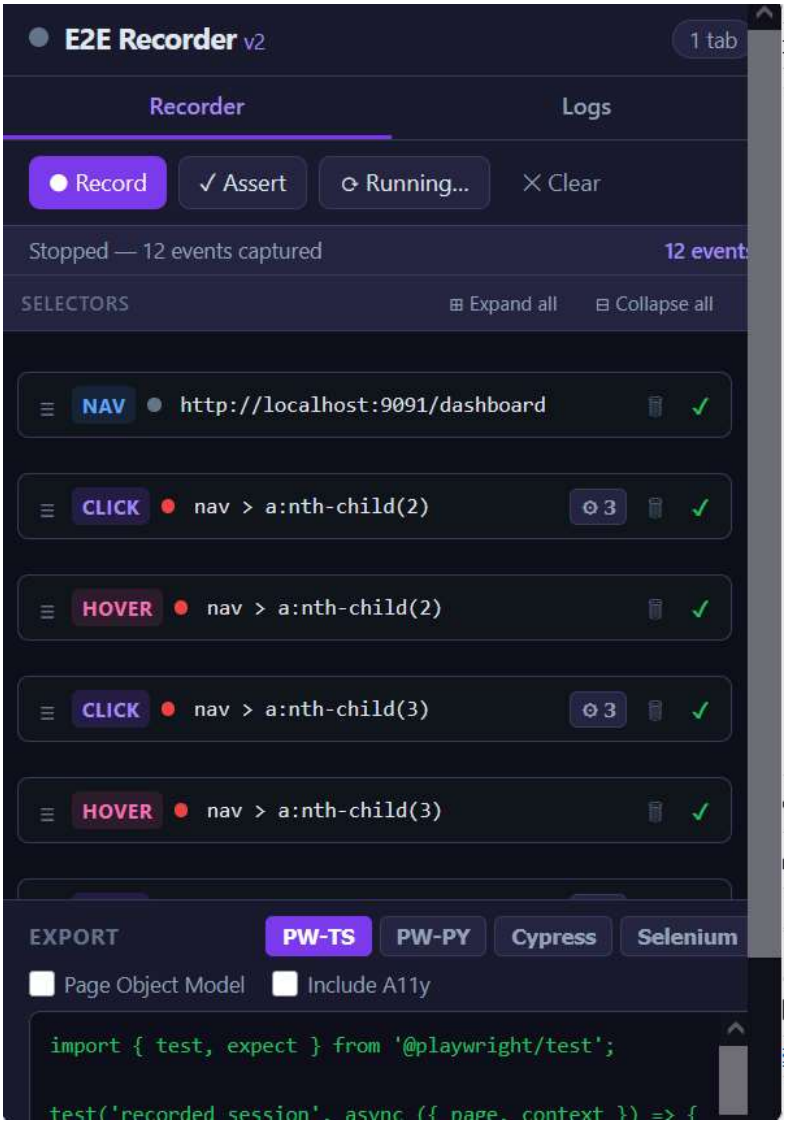
6. Build the package: run `build_Chrome.cmd` (Windows) or `build_Chrome.sh` (macOS / Linux). The output is `dist/e2e-recorder-v2-chrome.zip`.
7. Unzip the file into an empty folder, e.g. `dist/chrome-unpacked/`.
8. Open `chrome://extensions` (or `edge://extensions`).
9. Enable **Developer mode** using the toggle in the top-right corner.
10. Click **Load unpacked** and select the folder from step 2.
11. The E2E Recorder icon appears in the toolbar.

> **Note**
> Alternatively, you can upload `e2e-recorder-v2-chrome.zip` directly to the Chrome Web Store Developer Console if you are distributing the extension publicly.

3. Popup Overview

Click the E2E Recorder icon in the browser toolbar to open the popup. The popup is divided into two main tabs:

Tab	Purpose
Recorder	Record sessions, view and edit events, replay, and export code.
Logs	Real-time log messages from all extension contexts for debugging.



3.1 Recorder Tab Layout

From top to bottom the Recorder tab contains:

- Status bar — shows whether recording is active, and how many browser tabs are part of the current session.
- Control buttons — Record / Stop, and Assertion Mode.
- Editor toolbar — Expand All / Collapse All candidate panels (visible only when there are recorded events with selector alternatives).
- Event list — the chronological list of all captured interactions, with insert zones between items.
- Assertion suggestions — cards below the events that triggered a significant DOM change.

- Export section — framework selector, code output area, Copy and Download buttons.

4. Recording a Session

4.1 Starting and Stopping

12. Open the popup and click the green **Record** button.
13. Navigate to the page you want to test — the extension captures all interactions from that moment on.
14. Perform your test flow: click buttons, fill forms, navigate, etc.
15. Click **Stop Recording** (red button) when finished.

> **Visual indicator**
> A blinking red dot in the popup header confirms that recording is active. The browser tab title also shows an **[REC]** prefix.

4.2 Captured Event Types

The extension automatically captures the following interaction types:

Type	Badge	When captured
Click	CLICK	Any left-click on an interactive element.
Fill	FILL	Text typed into an input; emitted 400 ms after the last keystroke.
Key press	KEY	Non-printable keys: Enter, Tab, Escape, Arrow keys, Ctrl/Cmd combos.
Hover	HOVER	Cursor rests on an element for 600 ms and a DOM change is observed.
Scroll	SCROLL	Scroll inside a non-window container, or scroll that loads new content.
Drag & drop	DRAG	dragstart on source element; drop on target element.
File upload	FILE	File selection via <input type="file">. File name captured (not path).
Select option	SELECT	Native <select> value change.
Navigate	NAV	Page load, URL change, SPA route change.

4.3 Sensitive Data Masking

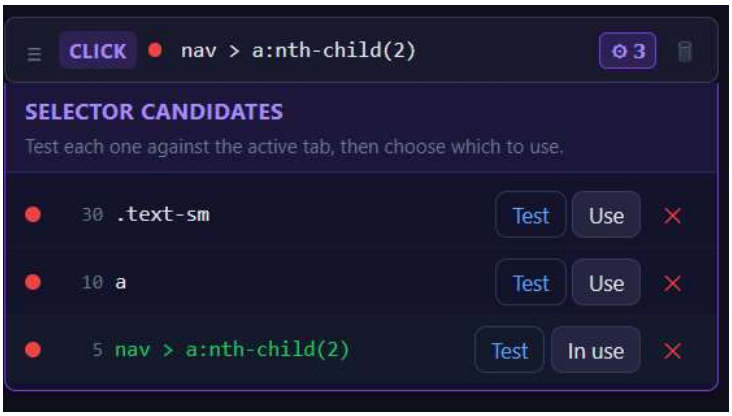
If a field is of type `password`, or its name / id matches patterns such as `api_key`, `token`, `cvv`, `secret`, the actual value is **never stored**. It is replaced with the placeholder `ENV_SECRET_PARAM` and the event is marked as masked. Replace this placeholder with an environment variable or fixture in the generated test code before running.

4.4 Traffic-Light Selector Inspector

While recording is active, a coloured outline appears over the element currently under the mouse cursor:

- Green outline — selector score ≥ 75. Highly stable (data-testid, id, aria-label).
- Amber outline — selector score 40–74. Moderately stable (name, placeholder, text).
- Red outline — selector score < 40. Fragile (class only, tag only, positional fallback).

This visual feedback lets you choose the best moment to click — e.g., wait for an element to acquire a `data-testid` attribute before interacting with it.



4.5 Multi-Tab Recording

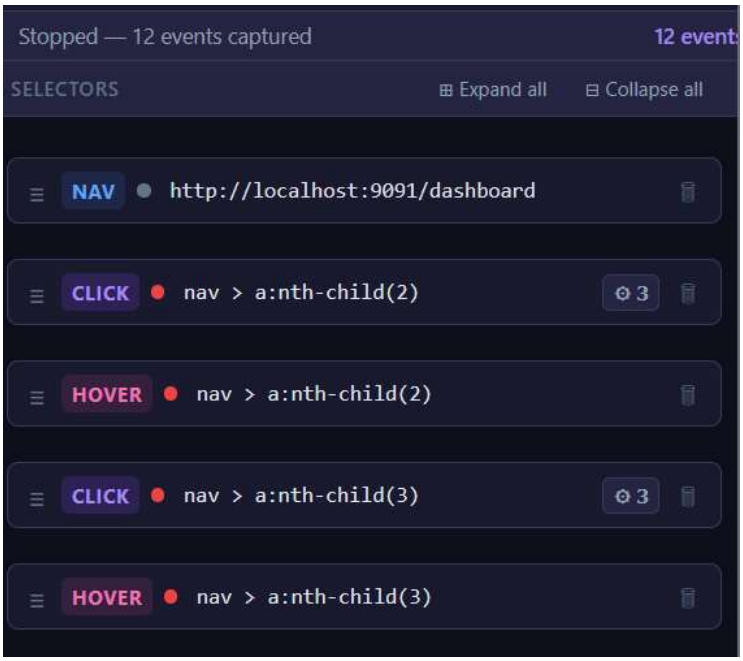
When a recorded action (such as "Sign in with Google") opens a new tab or pop-up window, the extension automatically detects it and continues recording in the new context. All events from every tab are merged into the same session, ordered by timestamp.

The number of active tabs in the session is shown in the popup header. When the secondary tab closes, recording resumes automatically in the primary tab.

5. The Event List

After recording, the event list shows every captured interaction as a row. Each row displays:

- `TYPE badge` — colour-coded action type.
- Selector or URL — the best selector found for the element.
- Value — for fill and keypress events.
- Health dot — green / amber / red, reflecting the selector score.
- Replay status — ✓ / ✗ / □ / – after running a replay.



5.1 Deleting Events

Click the trash icon on any event row to remove it from the session. The change is saved immediately.

5.2 Reordering Events

Drag the ☰ handle on the left side of a row up or down to change its position in the sequence. The timestamps are updated to maintain consistency.

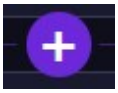
5.3 Editing a Selector Inline

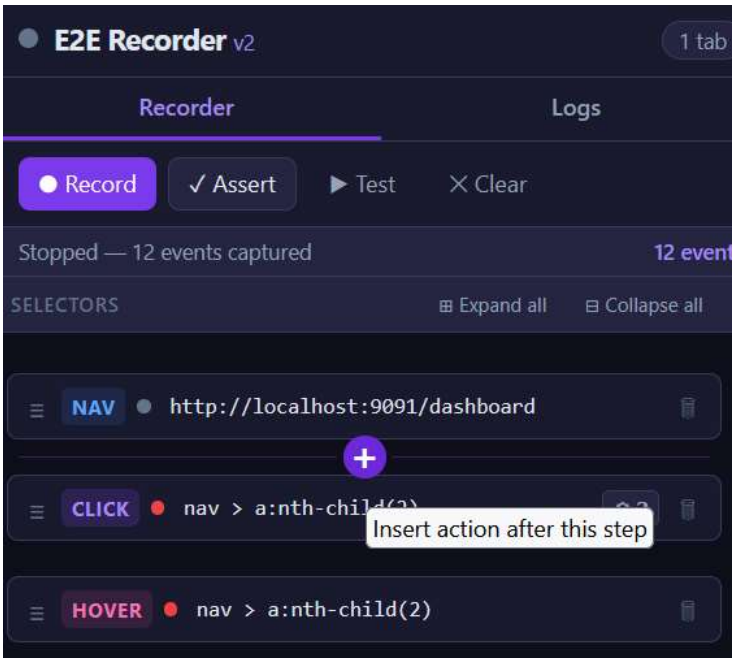
Click directly on the selector text in any row to make it editable. Type a new CSS selector and press Enter or click outside the field. The extension validates the selector against the currently active tab in real time:

- Green — selector matches exactly 1 element.
- Amber — selector matches 2–5 elements.
- Red — selector matches 0 or more than 5 elements.

5.4 Inserting New Steps

A thin + insertion zone appears between every pair of rows (hover to reveal it). Click it to open an inline form where you can manually add a new step:





Step type	Extra fields required
navigate	URL to navigate to.
click	CSS selector.
fill	CSS selector, value to type.
keypress	CSS selector, key name (e.g. Enter, Tab).

Step type	Extra fields required
hover	CSS selector.
scroll	CSS selector (or leave blank for window scroll), direction, amount.
selectOption	CSS selector of the <select>, option value.
wait	Duration in milliseconds.

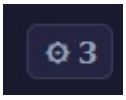
Click **Add** to insert the step, or **Cancel** to dismiss the form.

6. Selector Candidates

For every recorded interaction, the extension captures up to 6 alternative CSS selectors ranked by their stability score. This allows you to pick the most appropriate selector for your test environment instead of always accepting the automatic choice.

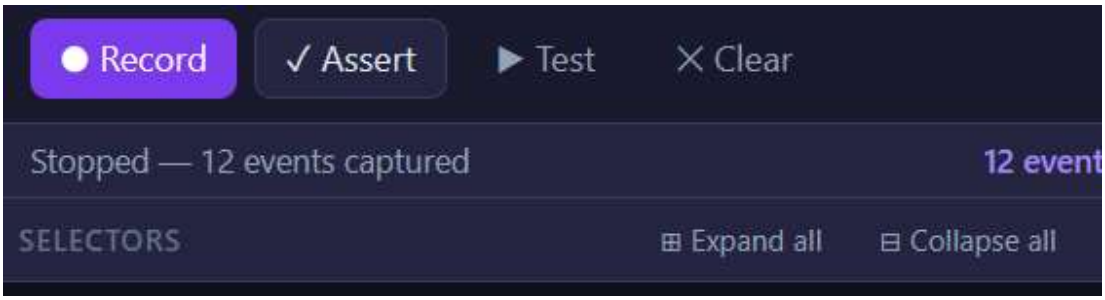
6.1 Opening the Candidates Panel

Each event row with available candidates shows a ⚙**Selectors** button on the right. Click it to expand the candidates panel below the row. Click it again to collapse.



Use the editor toolbar buttons to manage all panels at once:

- □ Expand all — opens every candidates panel simultaneously.
- □ Collapse all — closes every candidates panel simultaneously.



6.2 Candidate Score Dots

Each candidate row shows a coloured dot indicating its score tier:

- Green dot — score ≥ 70 (data-testid, id, aria-label, role, name).
- Amber dot — score 40–69 (placeholder, text-based).

- Red dot — score < 40 (class, tag, positional fallback).

6.3 Testing a Candidate

16. Click **Test** next to any candidate selector.
17. The extension sends the selector to the active tab and highlights every matching element.
18. The result label updates to show the match count:
    - `1 match` (green) — unique selector, ideal.
    - `N matches` (amber) — ambiguous, may interact with the wrong element.
    - `0 matches` (red) — element no longer on the page.

6.4 Using or Discarding a Candidate

- Use — click **Use** to replace the event's current selector with this candidate. The selector text in the parent row updates immediately.
- Discard — click **Discard** to remove this candidate from the list permanently.

> **Tip**
> Always test a candidate before clicking **Use**. A green • dot indicates a good score, but a real-time test confirms the selector still uniquely matches the live page.

7. Assertion Suggestions

After significant DOM changes following a click or fill action, the extension automatically suggests assertions. Suggested assertion cards appear below the event that triggered the DOM change.

7.1 Suggestion Types

Suggestion type	Generated assertion
urlChanged	Asserts that the page URL matches the expected value.
elementVisible	Asserts that a new element (e.g. a toast notification) is visible.
modalVisible	Asserts that a dialog (role="dialog") is present.
elementHidden	Asserts that an element (e.g. a loading spinner) has disappeared.

7.2 Accepting or Discarding Suggestions

Each suggestion card has two buttons:

- Accept (✓) — the assertion will be included in the exported code.
- Discard (✗) — the suggestion is removed and will not appear in the export.

Only **accepted** assertions are included in the generated test script.

8. Replay (Test Recording)

Use the replay feature to verify that your recorded selectors still work against the live page before exporting the script.

8.1 Running a Replay

19. Make sure the target page is open in the active tab (the URL should match the session's initial URL).
20. Click the □ **Test** button in the event editor.
21. Each event is executed in sequence. A spinner appears on the row being executed.
22. After all steps complete, every row shows a status indicator.

8.2 Status Indicators

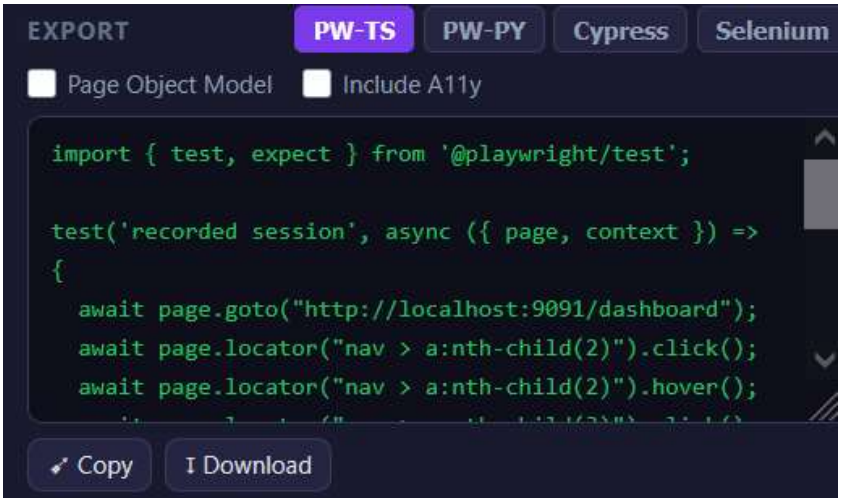
Symbol	Status	Meaning
✓	Passed	The step executed successfully.
✗	Failed	The selector was not found or the action could not be completed.
□	Running	The step is currently executing.
–	Skipped	Step type has no replay equivalent (e.g. navigate is handled separately).

Rows that fail are highlighted with a red outline. Review the selector for those steps and use the Candidates panel or inline edit to fix them before re-running.

> **Important**
> Replay simulates DOM interactions in the **live page** — it is not a fully isolated test run. Some actions may produce side-effects (form submissions, API calls). Use a staging environment or reset the application state between replays.

9. Exporting Test Scripts

The export section at the bottom of the Recorder tab generates ready-to-use test code from your recorded session.



9.1 Selecting a Framework

Use the framework tab bar to switch between:

- Playwright TS — Playwright test in TypeScript (recommended for new projects).
- Playwright PY — Playwright test in Python.
- Cypress — Cypress test in JavaScript.
- Selenium — Selenium WebDriver test in JavaScript (async/await).

9.2 Export Options

Option	Description
Page Object Model	When enabled, the export generates a separate class file per page / view in addition to the test file.
Include accessibility assertions	Adds optional aria-label / alt-text checks for interactive elements captured by click events.

9.3 Copying or Downloading

- Copy to Clipboard — copies the generated code to the system clipboard.
- Download File — downloads the generated code as a `.spec.ts` / `.spec.py` / `.spec.js` file. If Page Object Model is enabled and more than one file is generated, a ZIP archive is downloaded.

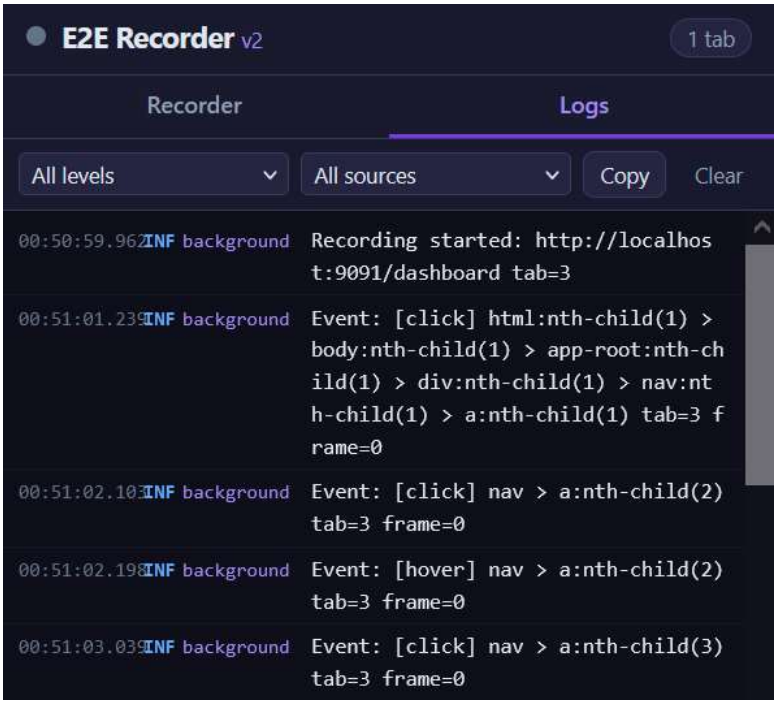
9.4 Sample: Playwright TypeScript

```
import { test, expect } from '@playwright/test';

test('Recorded test', async ({ page, context }) => {
  await page.goto('https://my-app.com/login');
  await page.fill('[name="username"]', 'sample_user');
  await page.fill('[name="password"]', ENV SECRET PARAM);
  await page.click('[data-testid="login-submit"]');
  await expect(page).toHaveURL('https://my-app.com/dashboard');
});
```

10. Logs Panel

The **Logs** tab in the popup captures diagnostic messages from all three extension contexts: background service worker, content scripts, and the popup itself. This is useful for troubleshooting recording problems.



10.1 Log Levels

Level	Usage
INFO	Normal operation messages.
WARN	Non-fatal issues (e.g. selector ambiguous).
ERROR	Failures that prevented an action from completing.
DEBUG	Verbose internal state messages.

10.2 Filtering Logs

- Use the **Level** dropdown to show only messages at or above a certain severity.
- Use the **Source** dropdown to filter by context: All, Background, Content, or Popup.

10.3 Copying and Clearing Logs

- Copy — copies all visible log entries to the clipboard as plain text.
- Clear — removes all stored log entries.

> **Note**

A red badge on the Logs tab shows the count of ERROR-level messages since the last time the panel was opened. The badge clears when you switch to the Logs tab.

11. Selector Scoring Reference

The selector engine evaluates each element against the following criteria and chooses the highest-scoring selector that uniquely identifies the element.

Attribute / Criterion	Score	Example selector
`data-testid` or `data-cy`	100	[data-testid="submit-btn"]
Stable id	85	#login-form
`aria-label`	75	[aria-label="Close dialog"]
`role`	72	[role="option"]
`name` attribute	70	[name="username"]
`placeholder`	65	[placeholder="Email address"]
Exact visible text	60	button:has-text("Login")
First CSS class	30	.btn-primary
Tag name only	10	button
Positional fallback (nth-child)	5	#form > :nth-child(3)

Dynamic-looking IDs or classes (containing 3+ consecutive digits, random hashes, or framework prefixes like `css-`, `Mui`, `chakra-`) receive a penalty of −50 points.

If no candidate achieves uniqueness after climbing 5 ancestor levels, the engine falls back to a positional `:nth-child` selector combined with the nearest stable ancestor. A selector is **always** returned.

12. Known Limitations

Limitation	Details
Cross-origin iframe highlighting	The traffic-light overlay does not appear inside cross-origin iframes (browser security restriction). Event capture still works because the content script is injected directly into the frame.
File upload paths	The real absolute file path cannot be captured. The generated code contains a PLACEHOLDER_REPLACE_WITH_REAL_PATH placeholder that must be replaced manually.
Selenium & Shadow DOM	Selenium has no native Shadow DOM piercing API. Generated code uses executeScript with a comment explaining the limitation.
Cypress & multi-tab	Cypress has limited multi-tab support. The generated script includes a warning comment and a cy.origin() suggestion.
Canvas / WebGL drag targets	If the drag target is a <canvas> element, only screen coordinates are captured (no CSS selector).
Firefox XPI signing	Installing an unsigned XPI via about:addons works only in Firefox Developer Edition or when xpinstall.signatures.required is set to false in about:config.

13. Troubleshooting

Symptom	Solution
Extension icon is missing from the toolbar.	Firefox: click the puzzle-piece icon to pin the extension. Chrome: click the Extensions icon and pin E2E Recorder.
No events are captured after clicking Record.	Check the Logs panel for ERROR messages. Reload the target tab and try again. Make sure the tab is not on a chrome:// or about: page.
Replay fails on all steps after a selector change.	The active tab may have navigated away from the recording URL. Reload the starting URL before replaying.
Selector Candidates panel is not visible.	The panel only appears for events that have more than one candidate. Simple elements (e.g. those with data-testid) may produce only one high-quality selector.
XPI build fails on Windows.	Run PowerShell as Administrator. Ensure the ExecutionPolicy allows scripts: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser.
Logs panel is empty.	Logs are stored in browser.storage.local. Clearing the extension storage (via browser DevTools) also clears the logs.

14. Quick Reference Card

Action	How to do it
Start recording	Open popup → click Record (green button)
Stop recording	Open popup → click Stop Recording (red button)
Delete an event	Click the □ icon on the event row
Reorder events	Drag the ≡ handle on any row
Edit a selector	Click the selector text in a row and type a new value
Insert a step between two events	Hover between rows to reveal the + zone, then click it
Open selector candidates	Click ⚙Selectors on any event row
Expand all candidate panels	Click □ Expand all in the editor toolbar
Test a candidate selector	Click Test next to the candidate in the panel
Accept an assertion suggestion	Click the ✓ button on the suggestion card
Replay the session	Click □ Test in the event editor
Export as Playwright TS	Select Playwright TS tab → Copy to Clipboard or Download File
Clear the session	Click Clear Session at the bottom of the popup
View extension logs	Click the Logs tab in the popup

E2E Recorder v2 is open-source software. For issues and source code, visit the project repository.